

软件工程基础

第8章软件部署、维护与演化

杨溢龙

北航软件学院

目　录

Contents

软件工程基础

软件部署、维护与演化

Part 1 软件部署

Part 2 软件维护

Part 3 软件演化

北京航空航天大學
BEIHANG UNIVERSITY



Part 1

软件部署

Part 1 软件部署

- 软件部署的概念
- 软件部署的原则
- 软件部署的分类
- 软件部署的方法
- 软件部署的工具

Part 1 软件部署

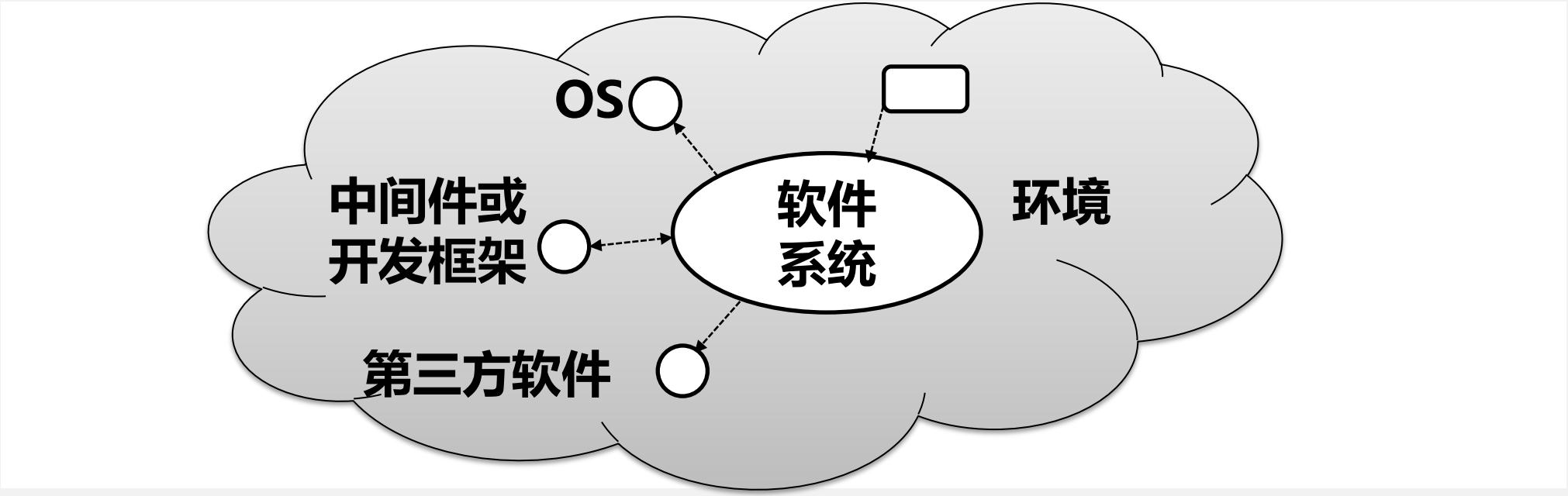
- 软件部署的概念
- 软件部署的原则
- 软件部署的分类
- 软件部署的方法
- 软件部署的工具

何为软件部署

- 软件部署是指将目标软件系统（包括软构件、配置文件、用户手册、帮助文档等）进行**收集、打包、安装、配置和发布**到**运行环境**的过程
- 涉及二方面的工作
  - ✓安装和配置运行环境
  - ✓安装和配置软件系统

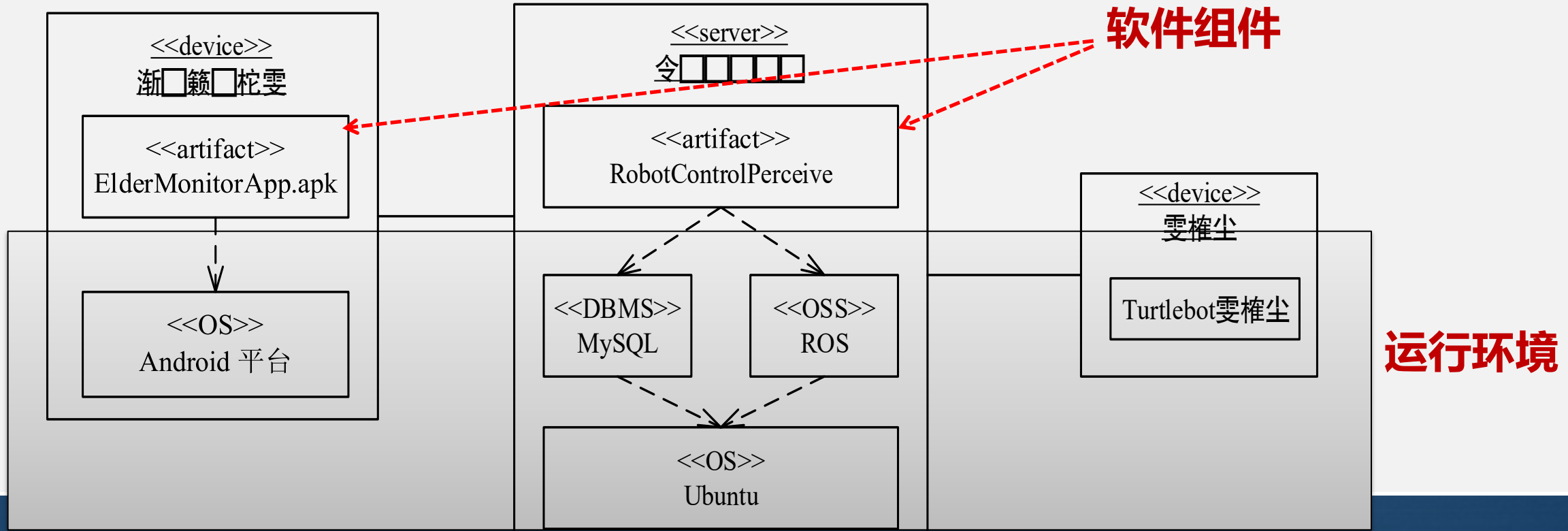
软件运行环境

- 软件运行所依赖的**上下文**，它为软件系统的运行提供必要的**基础服务和功能**、必须的数据和基本的**计算能力**



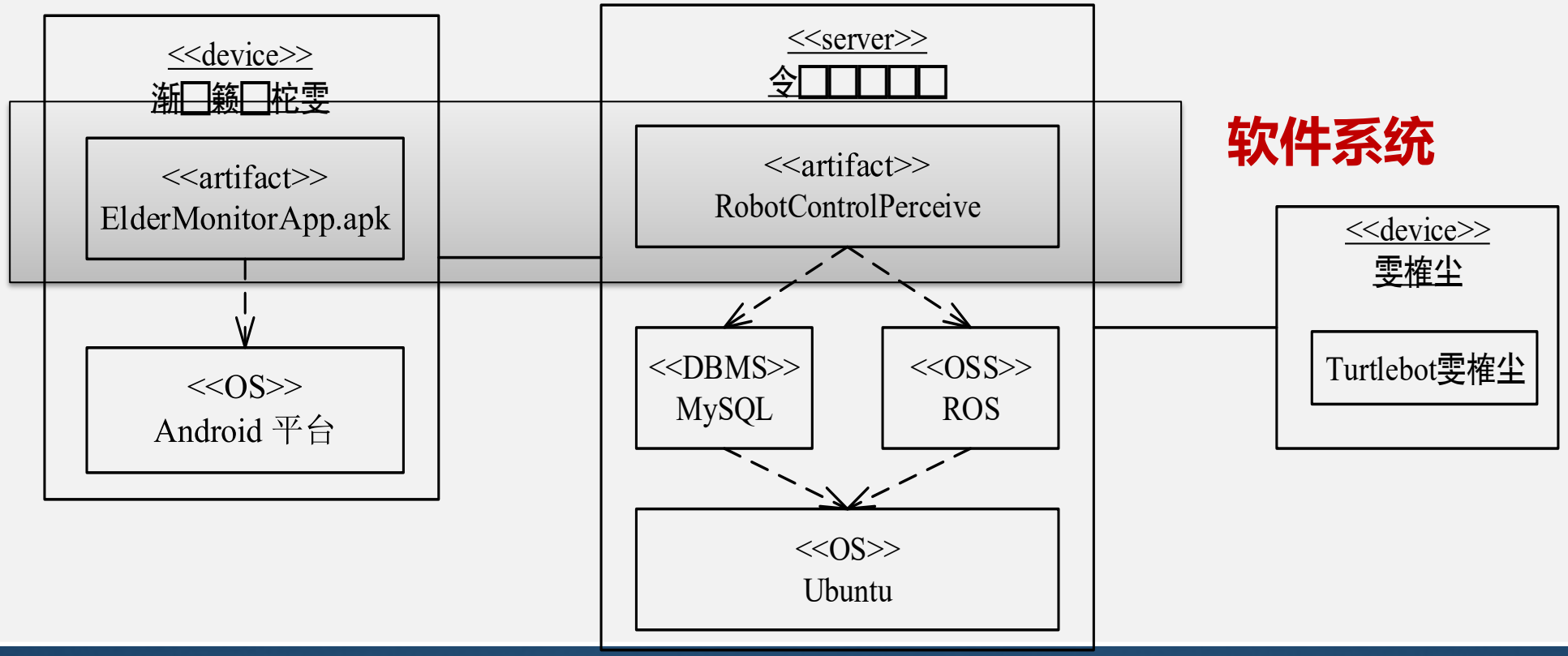
安装和配置运行环境

- 运行环境是目标软件系统运行赖以生存的**上下文**
- 在将软件系统部署到运行环境之前，软件开发工程师首先需要安装和配置好运行环境，包括**构成运行环境的各类软硬件系统以及它们之间的相关性**



安装和配置软件系统

- 收集和打包目标软件系统中需要安装的**软件要素**，然后将这些软件要素**安装到目标计算平台中**，并进行必要**配置**
  - ✓包括软件组件、所依赖软件包、软件文档和必要的数据

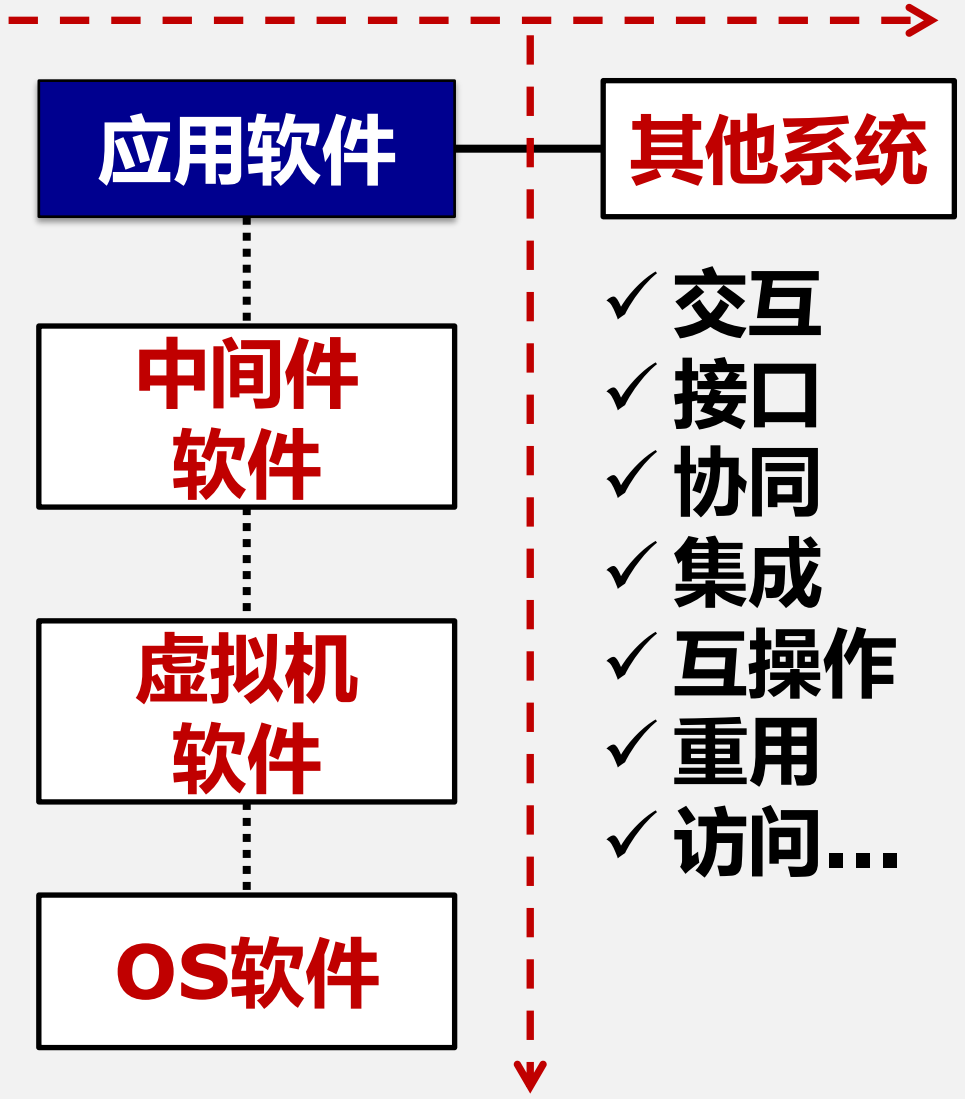


软件与其运行环境的关系

- **环境是软件赖以生存的场所**
  - ✓环境为软件运行提供各种要素，包括数据、计算、服务等等
- **软件需要与环境进行持续的交互**
  - ✓软件通过环境获得基础服务和计算能力，环境通过软件获得相应的运行进程和数据等
- **软件系统的运行环境可以表现为多种形式**
  - ✓既可以是物理和硬件设备（如计算机、服务器、机器人等），也可以是不同抽象层次的软件系统
- **软件系统的运行环境不仅包括纵向层次的基础软件及平台，还包括横向层次上与其运行相关的其他软件系统**

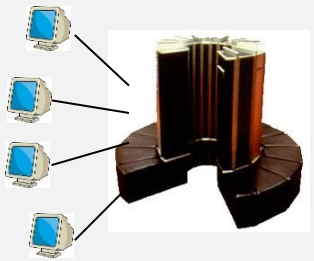
软件间的关系

- **纵向的层次性**
  - ✓依赖于支撑其运行的基础软件系统
- **横向的相关性**
  - ✓需要与其他的软件系统进行交互
- **软件部署需要考虑到**
  - ✓依赖的其他软件系统
  - ✓相关的其他软件系统



软件运行环境的变化

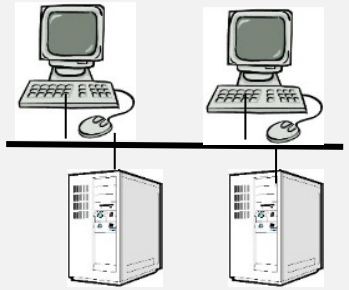
从前端的PC终端、可穿戴设备、智能手机到后端的云中心、高性能计算中心



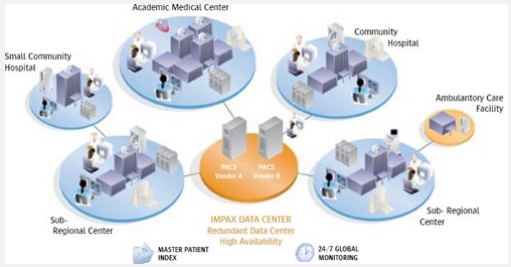
大中小型机的计算平台



基于PC的计算平台



基于局域网的分布计算平台



基于互联网的计算平台



无处不在的计算

从孤立、独立、局域和可控的计算环境到分布、开放、动态、难控、无处不在计算环境

Open Question

Points:10

设置

软件部署的主要问题和挑战在哪里?



Answer

Part 1 软件部署

- 软件部署的概念
- 软件部署的原则
- 软件部署的分类
- 软件部署的方法
- 软件部署的工具

软件部署的原则

❑最小化原则

- ✓只需安装、部署和配置支撑软件运行和服务提供的**最少软硬件要素**，以提高软件系统和**运行环境的精简性**，提升目标**软件系统的运行效率，减低运行和维护成本**

❑相关性原则

- ✓部署的运行环境和软件系统要素均与系统建设相关联，**剔除不想关的软硬件要素**，防止将无关的软件要素部署到计算平台之中，以简化软件系统的部署和配置，降低软件运行和维护的复杂度

❑适应性原则

- ✓当软件系统的运行**环境发生变化时**，**目标软件系统的部署也要随之发生变化**，以确保目标软件系统部署的灵活性，提高目标软件系统的健壮性，提升软件部署和运维的自动化程度

Part 2 软件部署

- 软件部署的概念
- 软件部署的原则
- 软件部署的分类
- 软件部署的方法
- 软件部署的工具

单机部署方式

- **将软件的各个要素（如可运行软构件、数据、文档等）集中部署到某个单一的计算设备上**
  - ✓软件的运行环境只依赖于单一的计算设施
  - ✓不同软构件之间**不存在网络通讯**
  - ✓计算设施不仅仅是指各种计算机，如个人计算机、笔记本电脑或服务器等，还包括智能手机、智能手环等嵌入式计算设施

- **典型示例**
  - ✓小米便签、闹钟和时钟、光盘刻录软件、扫雷游戏软件

分布式部署

- **将软件的各个要素（如可运行的软构件、数据和文档等）分散部署在多个计算设备上的部署方式**
  - ✓基于**C/S**的部署方式
  - ✓基于**客户端-应用服务器-数据库服务器**的部署方式
  - ✓基于**互联网**的软件部署方式

- **典型示例**
  - ✓空巢老人看护、Google搜索引擎、淘宝和中国铁路“12306”

Part 1 软件部署

- 软件部署的概念
- 软件部署的原则
- 软件部署的分类
- 软件部署的方法
- 软件部署的工具

软件部署方法

基于软件中间件、开发框架和容器部署的软件

基于操作系统部署的软件

软件中间件

软件开发框架

软件容器

操作系统层（如Windows、Linux、MacOS等计算机操作系统，及Android、IOS、鸿蒙等移动智能终端操作系统）

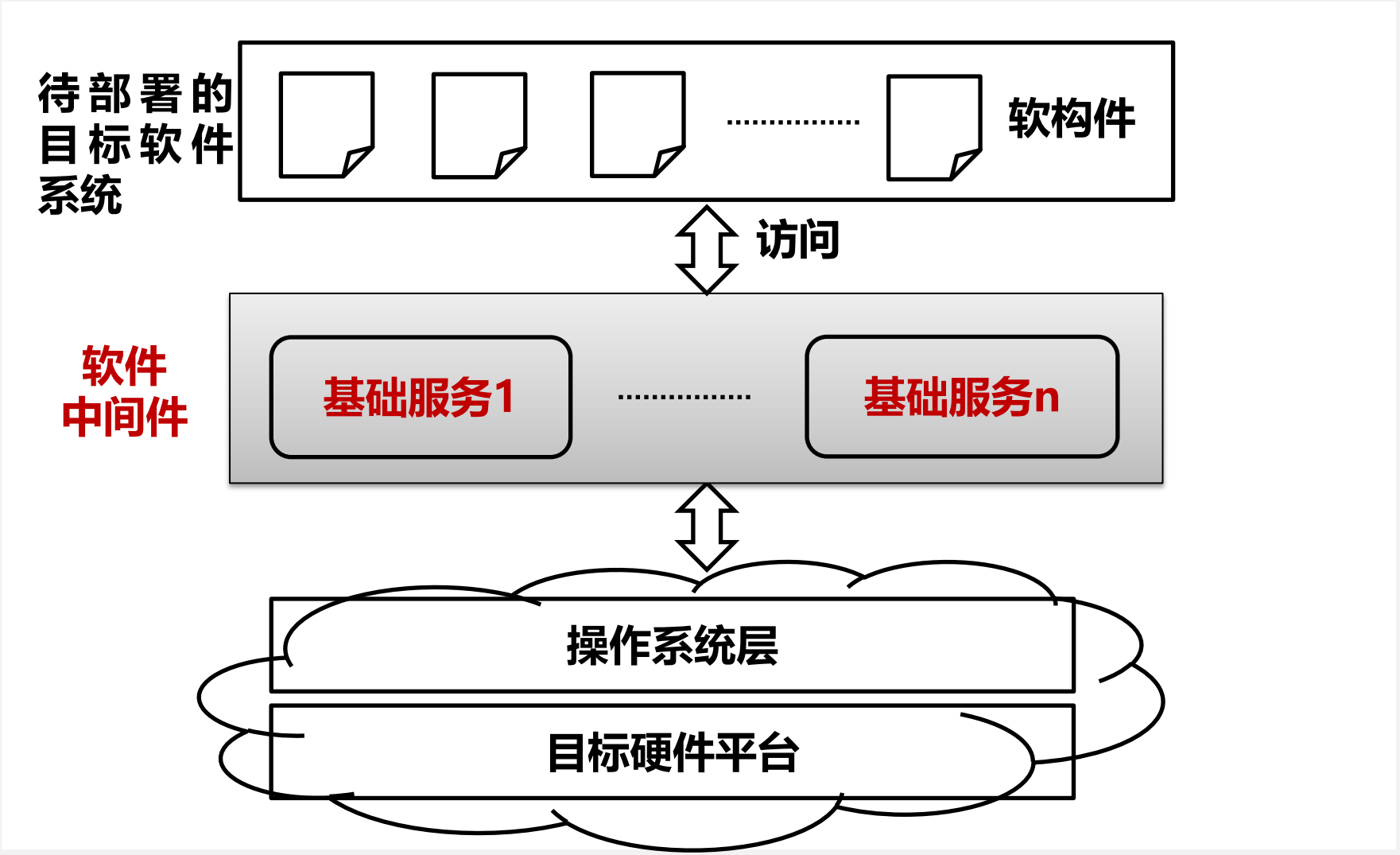
软件部署的目标平台（如计算机、服务器、移动智能终端等）

基于操作系统的部署

- **软件系统直接部署在目标计算设施的操作系统之上运行**
  - ✓软件的运行**仅依赖于计算设施上的操作系统及其提供的基础服务**（如文件操作和管理、窗口界面的创建和操作等）
  - ✓操作系统构成了软件运行的上下文环境

- **典型示例**
  - ✓**移动端APP软件部署**在移动端的操作系统之上，如智能手机、平板电脑等中的Android、IOS等
  - ✓**桌面端软件主要部署**在计算机上，常见的操作系统有Windows、Linux、MacOS等

基于软件中间件的部署



基于容器的部署

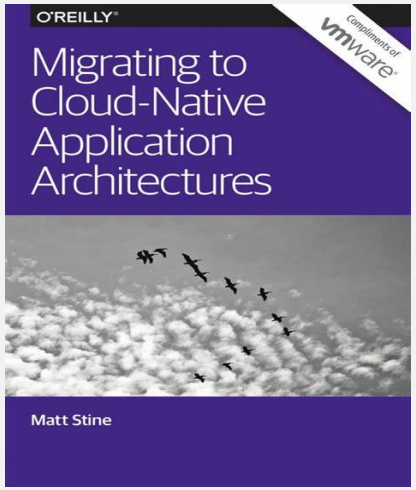
- 若用户仅需要使用一小部分资源运行一个简单应用程序时，虚拟机造成资源浪费，启动慢
- 急需一种比虚拟机更小的资源抽象粒度和资源分配单元

- **容器是一个视图隔离、资源可限制、具有独立文件系统的进程集合**
  - ✓ 容器作为软件运行的上下文
  - ✓ 容器具有独立文件系统，包括二进制文件、配置文件以及依赖

云原生 (Cloud Native)

- **Native**：表示应用程序从设计之初即考虑到云的环境
- 一种构建和运行应用程序的方法，**在云计算环境中**构建、部署和管理现代应用程序的软件方法
- 云原生应用充分利用云技术平台特有的**可扩展性、弹性**和**灵活性**优势





Pivotal

Pivotal Software Inc.
工程师架构师
Matt Stine
2015年出版的
《迁移到云原生应用架构》

https://www.oreilly.com/library/view/migrating-to-cloud-native/9781492047605/

生而为云：生在云上，长在云上，最大化地发挥云的能力

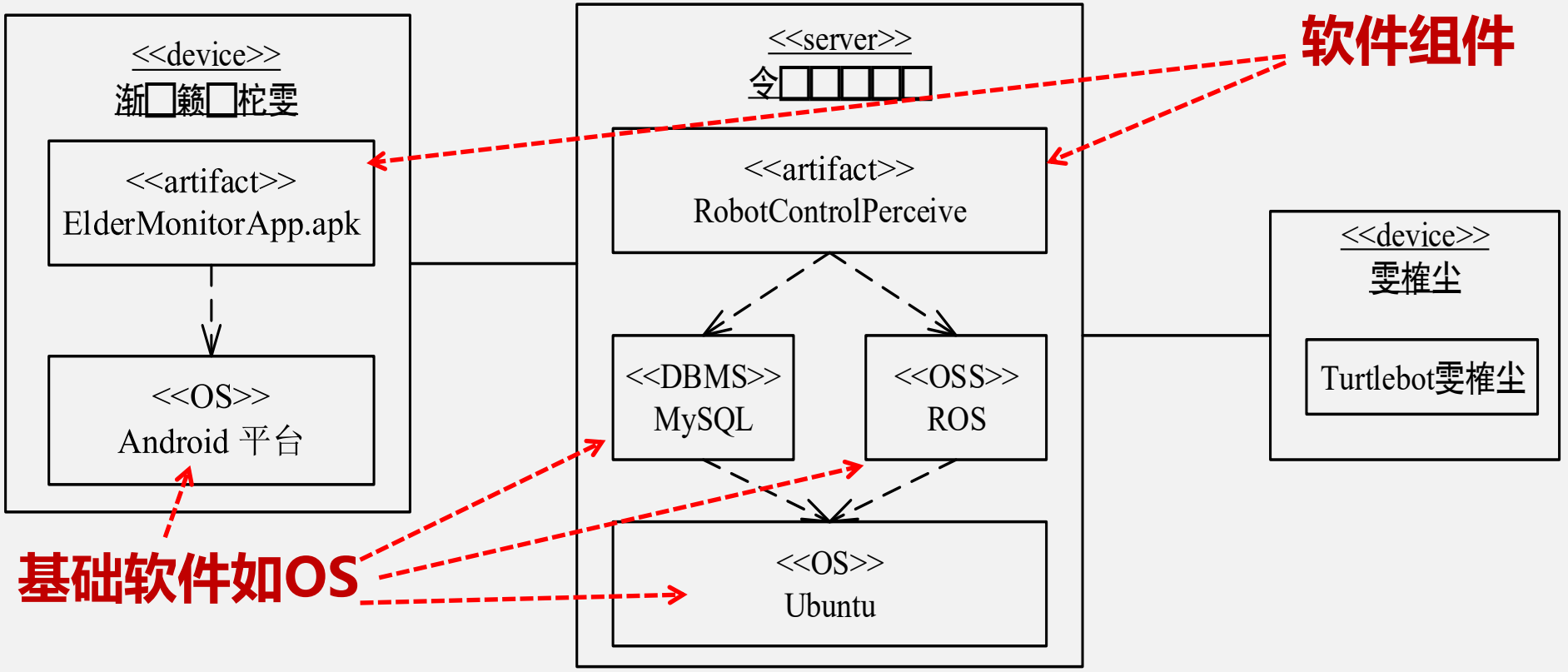
Part 1 软件部署

- 软件部署的概念
- 软件部署的原则
- 软件部署的分类
- 软件部署的方法
- 软件部署的工具

基于部署图规划部署

❑部署图

✓计算节点，通信方式，节点的运行环境，所要部署的软构件

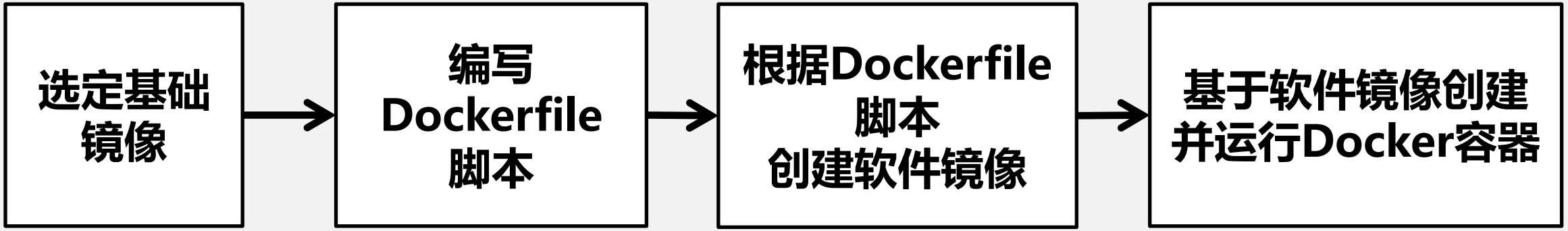


基于语言的部署工具：Fat jar

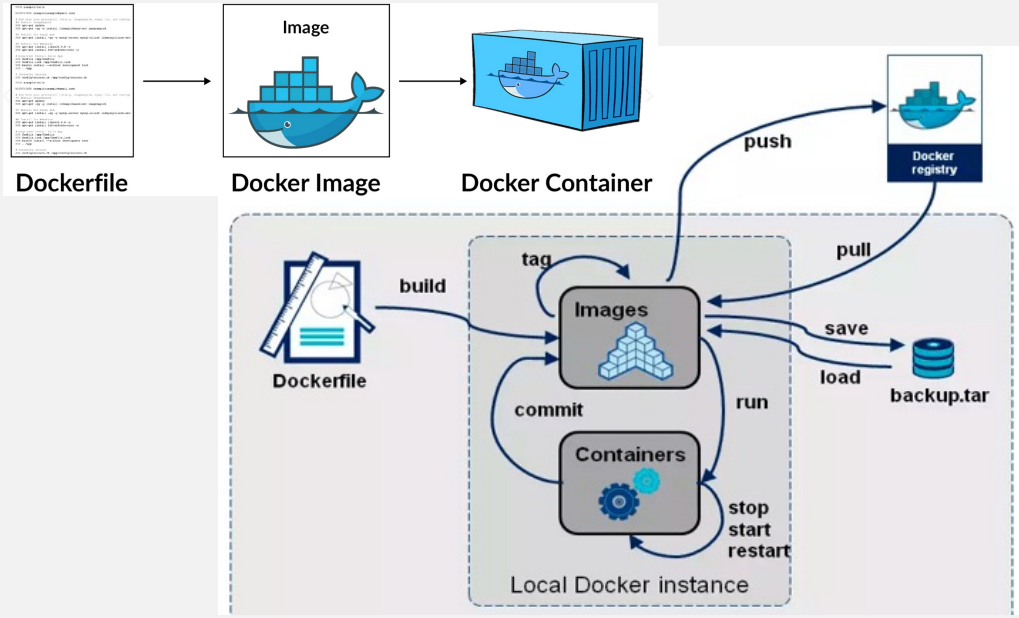
- **支持Java程序代码的打包、安装和部署**
  - ✓将Java程序的所有资源（包括源代码、调用的第三方库、图片、项目配置文件等）进行打包，生成可直接安装部署的jar文件
  - ✓能脱离Eclipse环境灵活部署到目标计算平台上
  - ✓是一个开源软件，托管在SourceForge开源社区中
  - ✓Eclipse集成开发环境的一个组件

基于容器的部署工具：Docker

容器镜像封装了应用及其依赖的运行环境，已经成为软件交付的标准



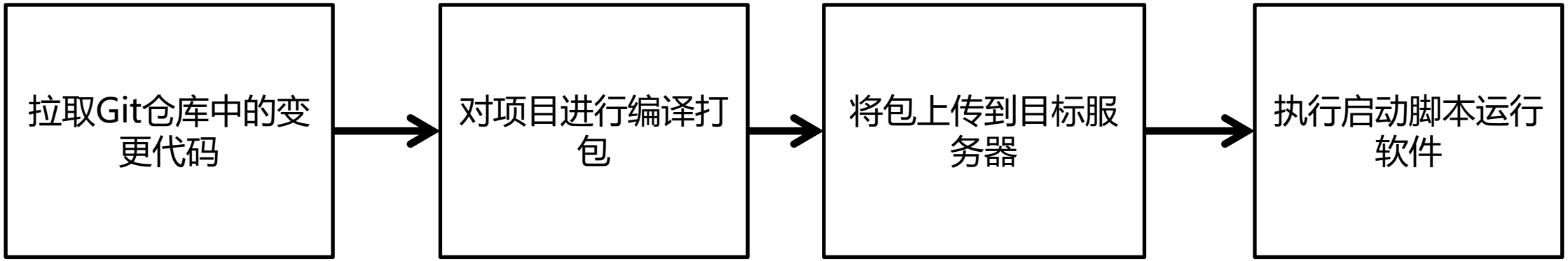

```
$ docker build
$ docker build .
Sending build context to Docker daemon  4.096kB
Step 1/7 : FROM ubuntu
 ---> 113a43faa138
Step 2/7 : RUN apt-get update
 ---> Using cache
 ---> 9fab80ce1a00
Step 3/7 : RUN apt-get install -y nginx
 ---> Using cache
 ---> f43971e1a4f3
Step 4/7 : COPY index.nginx-debian.html /var/www/html
 ---> 6d16b5929c14
Step 5/7 : EXPOSE 80 443
 ---> Running in f25e29f1f4d5
Removing intermediate container f25e29f1f4d5
 ---> 01ac6acf2470
Step 6/7 : COPY ["./start.sh", "/root/start.sh"]
 ---> d62a29c73a41
Step 7/7 : ENTRYPOINT /root/start.sh
 ---> Running in f0e33399d9f4
Removing intermediate container f0e33399d9f4
 ---> 53b851849970
Successfully built 53b851849970
```



基于流水线的部署工具：Jenkins

❑针对Java应用、实现持续集成和部署的软件工具

- ✓它允许其它的工具以插件的方式加入到CI/CD (Jenkins\Github Action)软件之中，为自动化的构建和部署提供多样化的服务



小结

- **软件需要部署和运行在特定的环境中**
  - ✓环境包括硬件、基础软件、中间件和其它软件系统等
- **软件部署是指将目标软件系统进行收集、打包、安装、配置和发布到运行环境的过程**
  - ✓单机部署和分布式部署
- **软件部署的方法多样化，取决于软件的运行环境**
  - ✓基于操作、基于中间件、基于开发框架、基于容器
- **支持软件部署的CASE工具**
  - ✓针对不同的编程语言和环境，提供不同的支持

北京航空航天大學
BEIHANG UNIVERSITY



Part 2

软件维护

Part 2 软件维护

- 软件维护的概念
- 软件维护的分类
- 软件维护的挑战
- 软件维护的过程

Part 2 软件维护

- 软件维护的概念
- 软件维护的分类
- 软件维护的挑战
- 软件维护的过程

物理设备的维护问题

- 物理设备在使用过程中会出现故障，需要进行维护





软件是否也会出故障？为什么？出了故障怎么办？



软件也需要进行维护

- **出故障，不可正常工作**
  - ✓潜在的**缺陷产生软件错误**
  - ✓需要对这些缺陷进行纠正
- **服务变化，需要升级**
  - ✓软件**需求发生了变化**
  - ✓需要增强软件的功能和服务
- **运行环境变化，需要适应**
  - ✓需要改变软件以在新的环境中运行

何为软件维护

- 软件在**交付使用**后，由于应用需求和环境变化以及自身问题，对软件系统进行**改造和调整**的过程
  - ✓软件自身问题，发现了一些新的缺陷
  - ✓运行环境变化，从Windows ➔ Linux
  - ✓软件需求变化，需要增加一些新的需求



小米便签软件的维护

- **设置便签访问密码**
- **解除便签访问密码**
- **根据密码访问便签**
- **改善程序代码质量**

软件维护的特点

❑同步性

- ✓软件维护需要与软件使用同步进行

❑周期长

- ✓软件维护周期会更长，一些软件会服役十几年甚至几十年的时间

❑费用高

- ✓维护成本高达总成本80%以上，维护费用是开发费用的**3倍以上**

❑难度大

- ✓充分理解待维护软件的架构、设计和代码，这极困难。尤其是在软件**设计文档缺失**的情况下，这一问题更为突出
- ✓50%-90%的时间被消耗在**理解程序**上

软件维护成本

- **维护成本不断增加**
  - ✓70年代： 35% - 40%
  - ✓80年代： 60%
  - ✓90年代： 75%
  - ✓如今：数据更高，80%

软件维护工作量$M = P + K * e^{(c-d)}$

- ✓ P=生产性工作量
- ✓ K=经验常数
- ✓ C=复杂度(设计好坏和文档完整程度)
- ✓ D=对欲维护软件的熟悉程度

- **软件维护工作量涉及二方面**
  - ✓助动性：用于理解代码功能，结构特征以及性能约束
  - ✓生产性：用于分析和评价、修改设计和代码
- **模型表明**
  - ✓如果没有好的软件开发方法或者软件开发人员不能参与维护，那么软件维护工作量会指数上升

Part 2 软件维护

- 软件维护的概念
- 软件维护的分类
- 软件维护的挑战
- 软件维护的过程

软件维护分类

- ✓ 软件维护的分类比较复杂，角度不同，分类的方法也不同，分类的结果也就不一样。
- ✓ 传统软件维护一般分为四大类，如下表所示：

序 号	维护的种类	维护的内容
1	纠错性维护	产品或项目中存在缺陷或错误，在测试和验收时未发现，到了使用过程中逐渐暴露出来，需要改正
2	适应性维护	这类维护是为了产品或项目适应变化了的硬件、系统软件的运行环境，如系统升级
3	完善性维护	这类维护是为了给软件系统增加一些新的功能，使产品或项目的功能更加完善与合理，又不至于对系统进行伤筋动骨的改造，这类维护占维护活动的大部分
4	预防性维护	这类维护是为了提高产品或项目的可靠性和可维护性，有利于系统的进一步改造或升级换代

纠正性维护

- **何为纠正性维护**
  - ✓纠正软件中的**缺陷和错误**
- **起因**
  - ✓用户在使用软件过程中一旦发现缺陷，他们会向开发人员提出纠正性维护的请求
- **目的**
  - ✓诊断和改正软件系统中潜藏的缺陷

为什么软件在使用时还有缺陷存在?



适应性维护

- **何为适应性维护?**
  - ✓对软件进行改造以便适应新的**运行环境和平台**
- **起因**
  - ✓软件运行于一定的环境(硬件、OS、网络等)之上，运行环境发展很快，出现了变化
- **目的**
  - ✓适应环境变化和发展而对软件进行维护

为什么软件需要适应环境的变化?



完善性维护

❑何为完善性维护？

✓对软件进行改造以**增加新的功能、修改已有的功能**

❑起因

✓在软件系统运行期间，用户可能要求增加新的功能、建议修改已有功能或提出其他改进意见

❑目的

✓满足用户日益增长的各种需求而对软件系统进行的改善和补充

小米便签维护中是否涉及完善性维护？做了哪些完善性工作



预防性维护

❑何为预防性维护？

✓对软件结构进行改造以便提高软件的**可靠性和可维护性**等

❑起因

✓为进一步改善软件系统的可维护性和可靠性，为以后的软件改进奠定基础的维护活动

❑目的

✓获取软件结构，重新改善软件结构

小米便签需要做预防性维护吗？



Part 2 软件维护

- 软件维护的概念
- 软件维护的分类
- 软件维护的挑战
- 软件维护的过程

软件维护需要解决的问题（1/3）

❑人员的问题

- ✓软件维护工程师认为软件维护**缺乏成就感**，从而影响他们的工作激情和投入
- ✓软件维护工程师**得不到足够的关注和重视**，从而影响对他们的支持和帮助
- ✓软件开发工程师**流动大**，软件维护工程师无法得到软件开发工程师的帮助
- ✓软件开发工程师**不愿意帮助**软件维护工程师

如何解决软件维护的人员问题？



软件维护需要解决的问题（2/3）

❑软件制品的问题

- ✓待维护的软件**不能提供软件文档**
- ✓待维护的软件**不能提供源程序代码**
- ✓待维护软件的源代码**可读性和可理解性差**，如缺乏必要的注释等
- ✓待维护软件的**文档可读性和可理解性差**，如啰嗦、语言不简练等
- ✓待维护软件的**文档不完整、不详实，漏掉**重要内容，缺少细节
- ✓待维护软件的**文档与其代码不一致**，影响对软件的理解和维护
- ✓要读懂待维护软件的文档和代码非常困难
- ✓软件制品的版本混乱，无法获得合适版本的软件制品

为了有效支持维护，软件开发工程中应注意哪些事项？



软件维护需要解决的问题（3/3）

❑维护副作用的问题

- ✓**代码副作用**，如修改或者删除程序、修改或者删除语句标号、修改逻辑符号等等。为此，软件维护工程师在变更代码时要非常慎重，切忌随意的修改代码
- ✓**数据副作用**，因修改信息结构而带来的不良后果，如局部和全局数据的再定义，记录或者文件格式的再定义等
- ✓**文档和模型副作用**，软件维护工程师在对程序代码进行修改的同时，必须**同步修改相关的模型和文档**，以确保模型与代码之间、文档与模型之间、文档与代码之间相互一致

改正旧的错误可能会导致新的错误



软件的可维护性

- ✓ 所谓**软件的可维护性**，就是维护人员理解、掌握和修改被维护软件的难易程度。可维护性的软件，它应具备下列四条性质，如下表所示。
- ✓ 符合上述四个条件的软件可维护性很好。反之，可维护性就差。

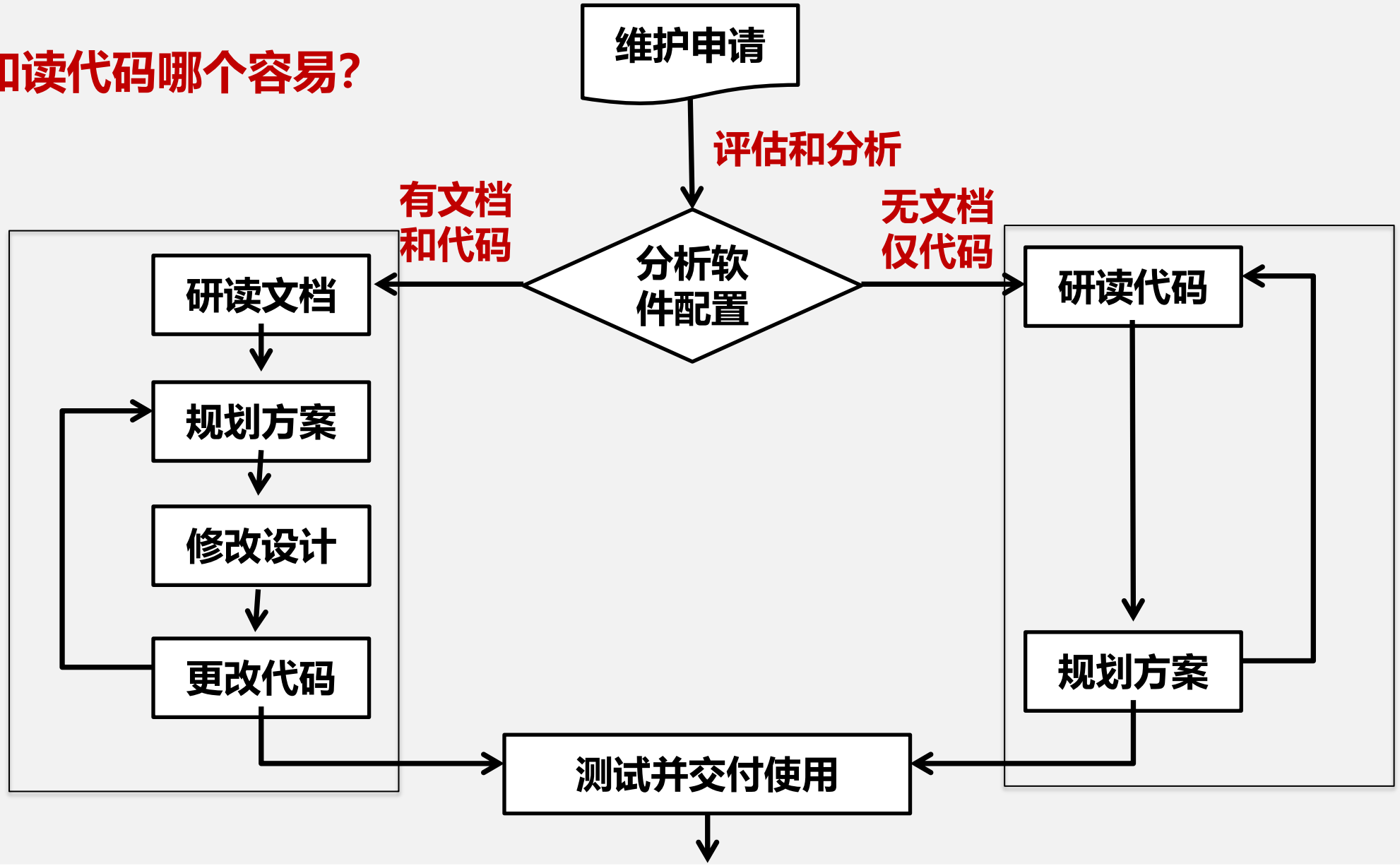
序号	可维护性名称	可维护性内容
1	可理解性	软件模块化、结构化，代码风格化，文档清晰化
2	可测试性	文档规范化，代码注释化，测试回归化
3	可修改性	模块间低耦合，高内聚，程序块的单入口和单出口，数据局部化，公用模块组件化
4	可移植性	例如用ODBC，ADO来屏蔽对数据库管理系统的依赖，用三层结构来简化对客户浏览层的维护

Part 2 软件维护

- 软件维护的概念
- 软件维护的分类
- 软件维护的挑战
- 软件维护的过程

软件维护过程

读文档和读代码哪个容易?



软件维护工作流程

流程步骤	流程内容
1	分类整理用户意见
2	提出维护申请
3	评审、审计、批准维护申请
4	修改需求文档
5	维护需求文档评审
6	修改设计文档
7	维护设计文档评审
8	修改源程序
9	回归测试
10	修改软件产品版本号
11	交付用户运行
12	收集用户反馈意见，准备进行新一轮维护活动，转向流程第1个步骤

维护管理文档

- ✓ （1）用户意见反馈表
- ✓ （2）用户意见分类整理表
- ✓ （3）维护申请单
- ✓ （4）维护文档评审报告
- ✓ （5）产品缺陷统计表
- ✓ （6）功能扩充统计表
- ✓ （7）未答复问题汇总表
- ✓ （8）未验证问题汇总表
- ✓ （9）已修改问题汇总表
- ✓ （10）已验证问题汇总表
- ✓ （11）维护费用统计表

维护文档

- ✓ 对于结构化维护来说，软件维护文档，就是对原来已有的分析文档、设计文档、实现文档、测试文档、用户指南进行修改，形成新的开发文档。新的开发文档的组织方式有两种格式：
- ✓ （1）格式1： 在先保存好原有的开发文档之后，**直接在原来已**
- ✓ **有的文档上面修改，修改后形成新的小版本号文档**，即在原来版本号中小圆点的右一位或右二位上加1。
- ✓ （2）格式2： **不直接在原来已有的文档上面修改，而是将修改的内容单独作为一个附录**，放在被修改文档的最后面，形成一份新的小版本号文档，即在原来版本号中小圆点的右一位或右二位上加1。

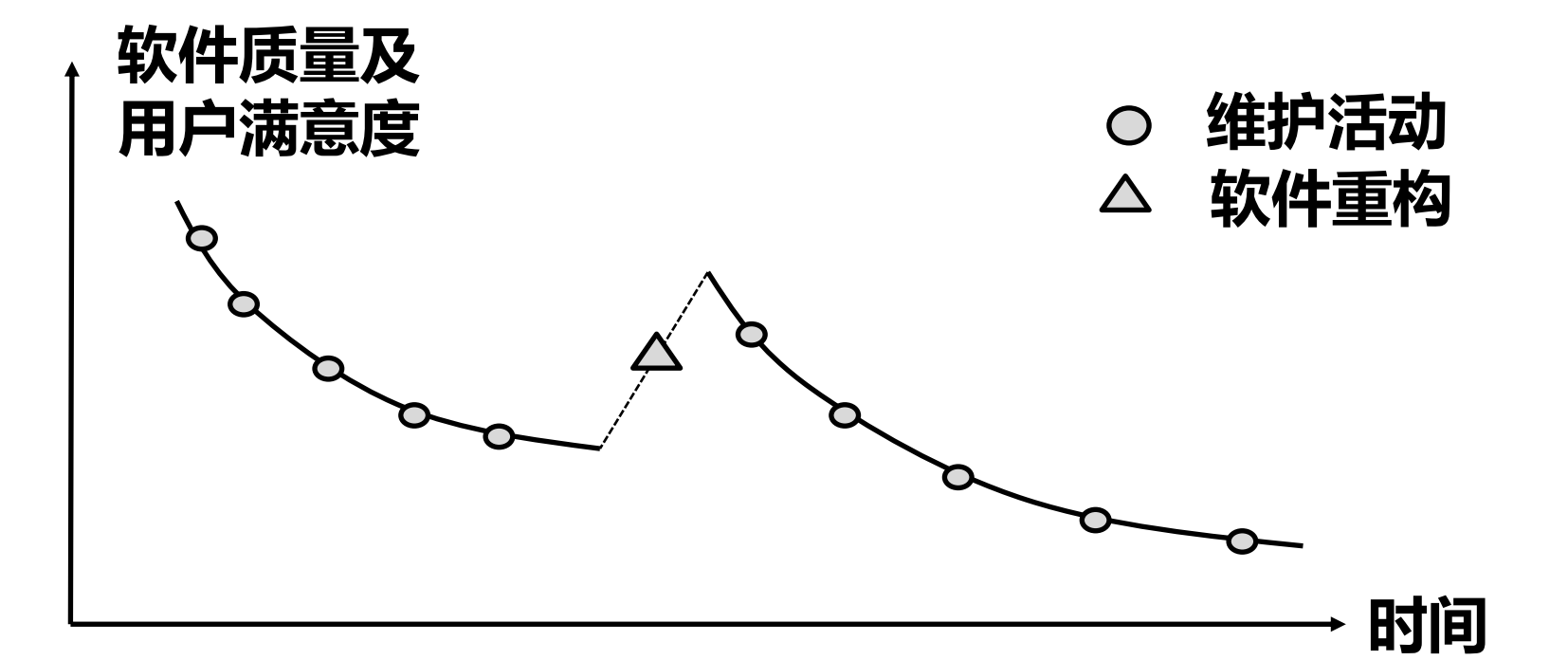


Part 3

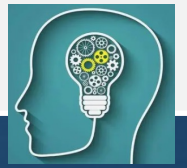
软件演化

何为软件逻辑老化

- 软件在维护和演化的过程中出现的**用户满意度降低、质量逐渐下降、变更成本不断上升**这样一种现象
  - ✓这些现象发生在逻辑层面，而非发生在物理层面



软件会出现物理老化现象吗?



软件逻辑老化的现象

❑质量下降

- ✓在对软件进行完善性维护的同时，尽管增加了新的功能，但也会**带来对软件架构的破坏**，从而**引入新的软件问题**，使得整个软件不易于维护，软件架构变得脆弱

❑变更成本增加

- ✓随着软件规模的不断增大、软件质量的下降，对软件进行变更的成本也会随之不断增加
- ✓**需要阅读更多的文档和代码**才能理解待维护的软件系统
- ✓软件系统架构的脆弱性意味着软件维护工程师不得不对软件进行**更多的“缝缝补补”**才能实现新的软件功能

❑用户满意度降低

- ✓随着对软件认识不断深入，用户会逐步发现软件中存在的缺陷
- ✓用户对软件的满意度会逐步降低

软件逻辑老化的原因

❑缺乏变更

- ✓当外部环境发生变化时，软件也应随之发生变化，进行必要的变更，否则软件就会进入老化
- ✓如果软件运行的操作系统已经发生了变化，软件就需要进行适应性的维护，否则就会被运行环境抛弃
- ✓**如果软件不能适应外部环境变化，缺乏必要变更，就会加速老化**

❑负面变更

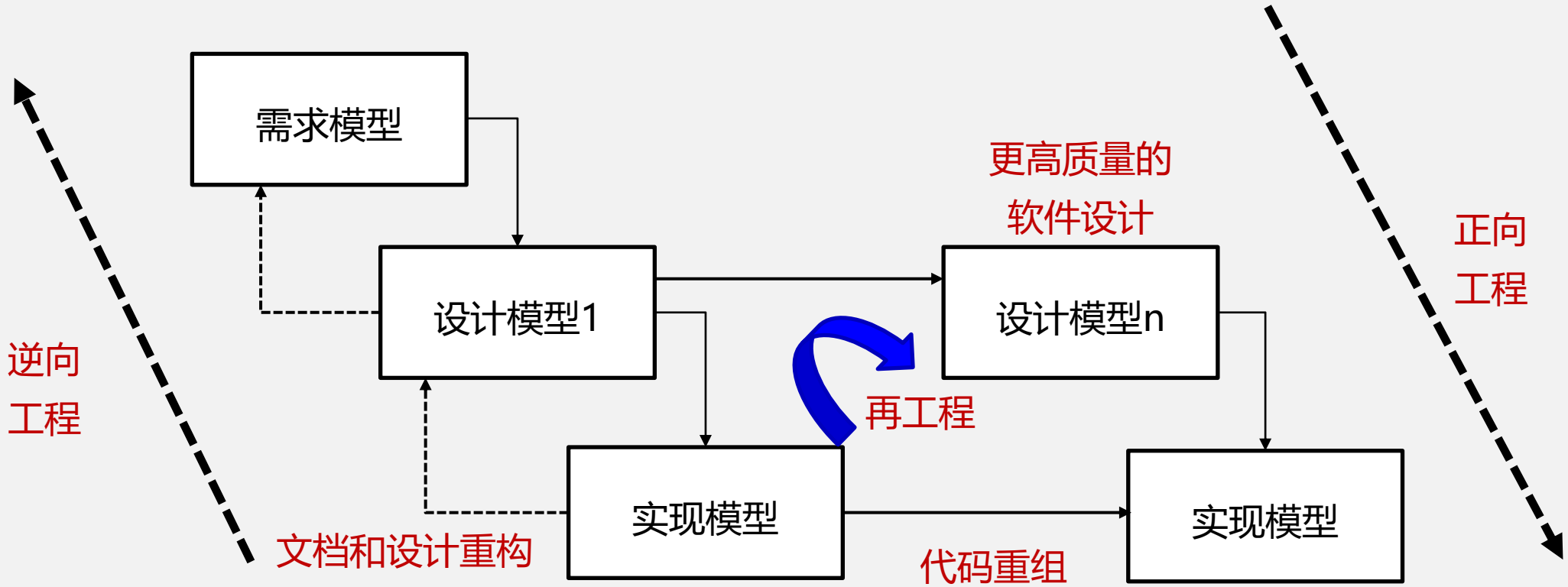
- ✓软件变更不总是积极和正面的，有时候它会**带来负面和消极影响**
- ✓通过变更引入了新的、更为严重的缺陷；破坏了软件结构，使得软件架构更为脆弱
- ✓**负面变更会破坏软件的结构和质量，进而增加维护的成本和难度**

软件演化及其特点

- **软件演化**是指针对软件的大规模功能增强和结构调整，以实现变化的软件需求，或者提高软件系统的质量
  - ✓ **功能增强粒度大**，软件演化针对的是粗粒度需求变化及功能增强
  - ✓ **主动应对变更**，基于对用户需求及其变化的理解，综合考虑软件各项功能实现的时间投入及开发成本，规划软件系统的整体演化，并以此开展功能增强等维护活动
  - ✓ **持续性**，预先规划好软件演化的路线图，完成当前软件演化工作后，软件维护团队随后将连续性的进入到另一项软件演化工作
  - ✓ **引发版本变化**，每一次演化结束后通常会产生一个新的软件版本

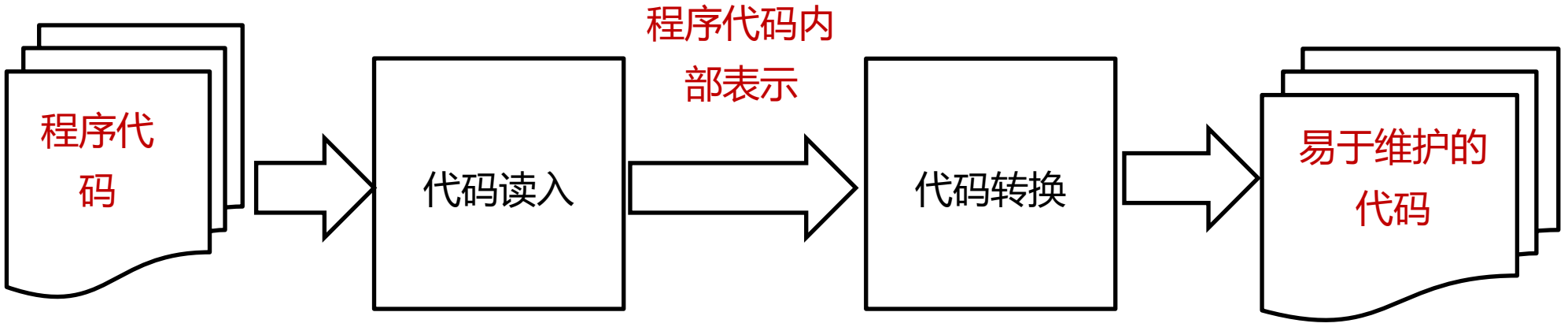
概念	功能增强粒度	应对变化方式	持续性或间隔性	版本变化
软件演化	粗粒度	主动	持续性	是
软件维护	细粒度	被动	间隔性	不一定

解决方法：逆向工程、重组、重构和再工程



代码重组

- 在不改变软件功能的前提下，对程序**代码**进行**重新组织**，使得重组后的代码具有更好的**可维护性**，能够有效支持对代码的变更



设计重构

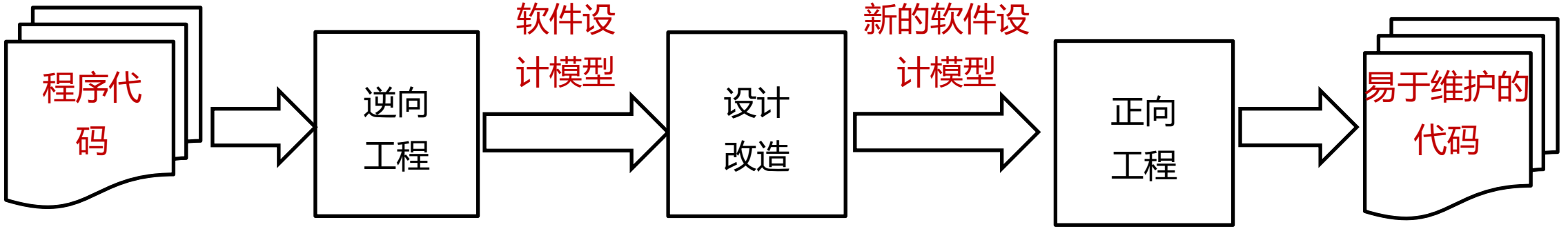
- 如果一个软件的设计文档缺失，软件文档与程序代码不一致、或者软件设计的内容不详实，那么软件维护工程师可以采用**设计重构的手段来获得软件设计方面的文档信息**
- 通过读入程序代码，理解和和分析代码中的变量使用、模块内部的封装、模块之间的调用或消息传递、程序的控制路径等方面的信息，**产生用自然语言或图形化信息所描述的软件设计文档**
- 是逆向工程的一种具体表现形式

逆向工程

- **基于低抽象层次软件制品，通过对其进行理解和分析，产生高抽象层次的软件制品**
  - ✓通过对程序代码进行逆向的分析，产生与代码相一致的设计模型和文档
  - ✓基于对程序代码和设计模型的理解，逆向分析出软件系统的需求模型和文档
- **典型应用场景**
  - ✓分析已有程序，寻求比源代码更高层次的抽象形式（如设计甚至需求）

再工程

- 通过分析和变更软件的架构，实现更高质量的软件系统的过程
- 再工程既包括逆向工程也包括正向工程



小结

- **多样化的软件维护**
  - ✓纠正性、完善性、适应性、预防性维护
- **持续性的软件演化**
  - ✓对软件的大规模功能增强和结构调整，以实现变化的软件需求，或者提高软件系统的质量
- **软件逻辑老化**
  - ✓解决软件逻辑老化的有效方法之一就是对软件进行重构
- **软件维护技术**
  - ✓逆向工程、再工程、代码重组、设计重构
  - ✓软件维护会带来新问题，同样需要关注质量因素，进行必要质量保证

北京航空航天大學
BEIHANG UNIVERSITY

Thank you

北京航空航天大学